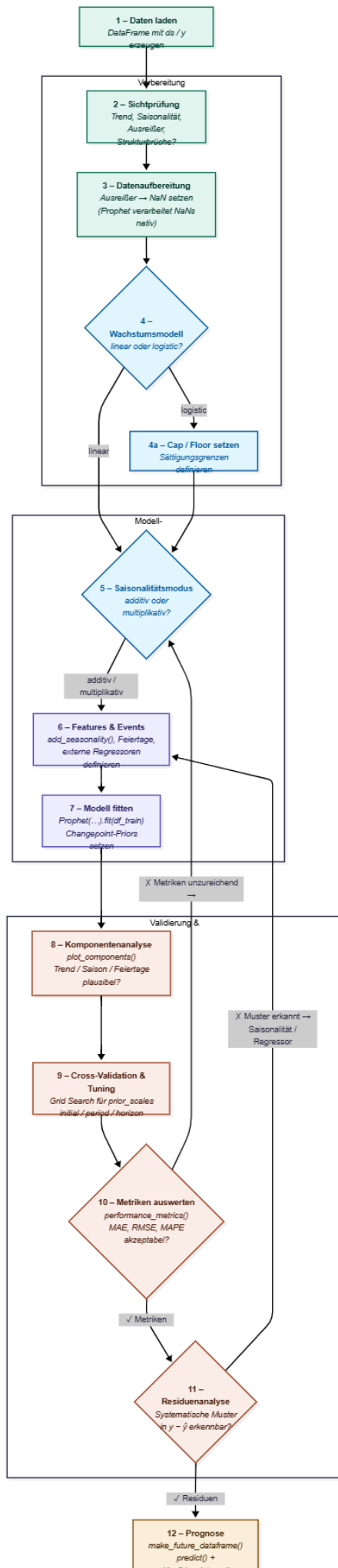


Der Prophet-Workflow

Ein Leitfaden zur Zeitreihenprognose mit Facebook Prophet

Dieses Skript erläutert den prototypischen 12-stufigen Prophet-Workflow, den Sie im zugehörigen Jupyter-Notebook vorfinden. Es richtet sich an Studierende, die bereits mit klassischen Zeitreihenverfahren (insbesondere SARIMA) vertraut sind und nun das moderne, praxisnahe Prophet-Verfahren kennenlernen.

Das Skript ist in vier Teile gegliedert: Teil A führt in die theoretischen Grundlagen von Prophet ein, Teil B beschreibt das erwartete Datenformat, Teil C erklärt die zwölf Arbeitsschritte im Detail, und Teil D fasst Entscheidungen und Stellschrauben tabellarisch zusammen.



Teil A — Theoretische Grundlagen

1. Was ist Prophet?

Prophet ist ein Open-Source-Verfahren zur Zeitreihenprognose, das 2017 vom Forschungsteam von Facebook (heute Meta) veröffentlicht wurde. Entwickelt wurde es für genau jene Zeitreihen, die im Unternehmensalltag typischerweise anfallen: tägliche Nutzerzahlen, monatliche Umsätze, Auslastungsprofile, Absatzzahlen — also Daten mit ausgeprägter Saisonalität, erkennbaren Feiertagseffekten und gelegentlichen Ausreißern.

Die Designphilosophie unterscheidet Prophet von klassischen Verfahren wie SARIMA: Während SARIMA mathematisch elegant ist, aber an strenge Voraussetzungen gebunden bleibt (Stationarität, gleichmäßige Zeitabstände, saubere Daten), verfolgt Prophet einen bewusst pragmatischen Ansatz. Das Modell soll auch von Anwenderinnen und Anwendern ohne tiefe statistische Vorbildung sinnvoll bedient werden können und dabei robust gegenüber den Unebenheiten realer Datensätze sein.

2. Die Kernidee — Zerlegung in interpretierbare Bausteine

Prophet beruht auf einer einfachen Grundidee: Jede Zeitreihe wird als Summe mehrerer leicht verständlicher Komponenten betrachtet. Die additive Grundgleichung lautet:

$$y(t) = g(t) + s(t) + h(t) + \varepsilon(t)$$

y(t) bezeichnet den beobachteten Wert zum Zeitpunkt t . Die drei strukturierten Komponenten sind: **g(t)** als Trend (das langfristige Niveau),

s(t) als Saisonalität (wiederkehrende Muster), und

h(t) als Feiertagskomponente (einmalige, aber kalendarisch wiederkehrende Effekte).

ε(t) ist der Restfehler — alles, was die drei strukturierten Bausteine nicht erklären können.

Der didaktische Vorteil dieser Zerlegung: Nach dem Training lässt sich jede Komponente einzeln visualisieren und auf Plausibilität prüfen. Eine Prophet-Prognose ist damit keine undurchschaubare Black Box, sondern eine nachvollziehbare Zerlegung mit interpretierbaren Teilen.

Additiv oder multiplikativ?

Manchmal wachsen saisonale Schwankungen mit dem Niveau der Zeitreihe mit. Hat eine Fluglinie im Sommer regelmäßig doppelt so viele Passagiere wie im Winter und bleibt dieser Faktor auch bei wachsendem Gesamtvolumen konstant, so ist die Saisonalität multiplikativ. Prophet rechnet dann intern:

$$y(t) = g(t) \cdot (1 + s(t) + h(t)) + \varepsilon(t)$$

Faustregel: Bleibt die Schwankungsbreite im Zeitverlauf konstant → additiv. Wächst sie mit dem Trend → multiplikativ. In SARIMA hätten Sie dieses Problem traditionell durch eine Log-Transformation adressiert — Prophet bietet es einfach als Schalter an.

3. Die Trendkomponente $g(t)$

Prophet unterstützt zwei Trendmodelle. Das lineare Modell ist der Standardfall und unterstellt einen langfristig geradlinigen Verlauf — allerdings mit der Möglichkeit, dass sich die Steigung an bestimmten Punkten der Zeitreihe ändern darf. Das logistische Modell hingegen bildet eine S-Kurve ab und wird eingesetzt, wenn eine natürliche Sättigungsgrenze bekannt ist (etwa Marktdurchdringung oder Kapazitätsgrenzen). In diesem Fall muss die Obergrenze (cap) und optional eine Untergrenze (floor) vom Anwender vorgegeben werden — das ist reines Domänenwissen und kann nicht aus den Daten allein abgeleitet werden.

Changepoints — automatische Erkennung von Trendwenden

Ein zentrales Konzept ist der Changepoint: ein Zeitpunkt, an dem sich die Steigung des Trends ändert. Prophet platziert automatisch typischerweise 25 potenzielle Changepoints gleichmäßig über die ersten 80 % der Zeitreihe und lässt das Modell entscheiden, welche davon tatsächlich genutzt werden. Wie großzügig Prophet dabei vorgeht, regelt der Hyperparameter

changepoint_prior_scale: Hohe Werte erlauben viele und starke Trendänderungen (flexibles, aber potenziell überangepasstes Modell); niedrige Werte erzwingen einen glatteren Trend (robust, aber möglicherweise zu steif).

4. Die Saisonalitätskomponente $s(t)$ — das Fourier-Prinzip

Die Modellierung der Saisonalität ist mathematisch der interessanteste Teil von Prophet. Die zugrunde liegende Idee lässt sich an einer Analogie aus der Akustik leicht verstehen: Ein musikalischer Akkord klingt komplex, lässt sich aber als Überlagerung einzelner, einfacher Sinus-Schwingungen schreiben. Jede dieser Schwingungen hat eine bestimmte Frequenz und eine bestimmte Amplitude (Lautstärke). Addiert man genügend viele Sinuswellen mit jeweils passender Frequenz und Amplitude, lässt sich praktisch jedes periodische Signal nachbauen. Genau dieses mathematische Prinzip — die sogenannte Fourier-Reihe — nutzt Prophet zur Modellierung saisonaler Muster.

Für eine Saisonalität mit Periodendauer P (also etwa 365,25 Tage bei Jahresmustern oder 7 Tagen bei Wochenmustern) modelliert Prophet die Saisonalitätskomponente als:

$$s(t) = \sum [a_n \cdot \cos(2\pi \cdot n \cdot t / P) + b_n \cdot \sin(2\pi \cdot n \cdot t / P)]$$

wobei die Summe über $n = 1$ bis N läuft. N heißt Fourier-Ordnung und steuert, wie fein das saisonale Muster modelliert wird:

- **Niedriges N (z. B. 3):** Nur langsame, glatte Sinuswellen werden berücksichtigt. Das Modell zeichnet eine weiche Jahreskurve.
- **Hohes N (z. B. 20):** Auch schnelle Schwingungen werden zugelassen. Das Modell kann feine Zacken nachbilden, läuft aber Gefahr, Rauschen mitzulernen (Overfitting).

Die Standardwerte von Prophet sind $N = 10$ für Jahressaisonalität und $N = 3$ für Wochensaisonalität. Die Koeffizienten a_n und b_n werden automatisch aus den Daten geschätzt — der Anwender muss lediglich die Fourier-Ordnung (wenn überhaupt) justieren.

5. Feiertage und externe Regressoren

Neben Trend und Saisonalität kennt Prophet zwei weitere Mechanismen, um domänenspezifisches Wissen in das Modell einzuspeisen.

Feiertage ($h(t)$): Kalendereffekte mit bekannten Daten — Weihnachten, Ostern, Black Friday, Produktlaunches. Prophet bringt eingebaute Kalender für viele Länder mit, erlaubt aber auch benutzerdefinierte Feiertagslisten. Optional kann ein Fenster rund um den Feiertag definiert werden (z. B. der 24. bis 26. Dezember).

Externe Regressoren: Nicht jeder Einflussfaktor ist periodisch. Temperatur, Werbebudget, Wirtschaftsindikatoren — solche Größen sind externe Regressoren und werden mittels `add_regressor()` in das Modell eingebunden. Wichtig: Für die Prognose müssen die Werte des Regressors auch im zukünftigen Zeitraum bekannt oder geschätzt sein.

6. Abgrenzung zu SARIMA

Da Sie SARIMA bereits kennen, lohnt sich ein direkter Vergleich der beiden Verfahren. Die folgende Tabelle stellt die wesentlichen Unterschiede gegenüber:

Aspekt	SARIMA	Prophet
Stationaritätsannahme	Erforderlich (Differenzieren nötig)	Nicht erforderlich
Gleichmäßige Zeitabstände	Zwingend erforderlich	Nicht zwingend, Lücken erlaubt
Umgang mit NaN/Ausreißern	Manuelle Vorverarbeitung nötig	Nativ unterstützt
Modellwahl	Ordnungen p, d, q, P, D, Q manuell bestimmen	Wenige qualitative Entscheidungen
Varianzstabilisierung	Üblicherweise via Log-Transformation	Schalter additiv / multiplikativ
Ausgabe	Ein zusammenhängendes Modell	Dekomposition in sichtbare Bausteine
Interpretierbarkeit	Parameter schwer intuitiv zu lesen	Jede Komponente einzeln plotbar

Wichtig: Diese Gegenüberstellung ist kein Plädoyer gegen SARIMA. Für manche Fragestellungen — insbesondere bei sauberen, stationären Reihen mit klarer autoregressiver Struktur — ist SARIMA weiterhin erste Wahl. Prophet ist dann im Vorteil, wenn Saisonalität dominiert, Feiertagseffekte eine Rolle spielen und die Datenqualität nicht makellos ist — also im typischen Business-Alltag.

Teil B — Das Datenformat

Die Schlichtheit des erforderlichen Datenformats ist ein Markenzeichen von Prophet. Während andere Verfahren oft mit Indizes, Frequenzobjekten und Typ-Konvertierungen ringen, erwartet Prophet einen pandas-DataFrame mit exakt zwei Pflichtspalten:

Spalte	Datentyp	Bedeutung
ds	datetime	Zeitstempel der Beobachtung (»datestamp«)
y	float / int	Messwert zum Zeitpunkt ds (die Zielgröße)

Jede weitere Spalte im DataFrame wird nur dann benötigt, wenn Sie sie explizit nutzen möchten: `cap` und `floor` bei logistischem Wachstum, beliebige weitere Spalten als externe Regressoren.

Minimalbeispiel

ds	y
2024-01-01	112.0
2024-02-01	118.0
2024-03-01	132.0
...	

Frequenz und Zeitabstände

Prophet kommt mit einer Vielzahl von Frequenzen zurecht: stündlich, täglich, monatlich, quartalsweise. Gleichmäßige Zeitabstände sind nicht zwingend erforderlich, erleichtern aber die Interpretation der saisonalen Komponenten. Für die Zukunftsprognose wird die Frequenz an

`make_future_dataframe(..., freq=...)` übergeben — üblich sind MS (Monatsanfang), D (Tag), H (Stunde), QS (Quartalsanfang).

Fehlende Werte und Ausreißer

Ein oft übersehener, aber praktisch hochrelevanter Vorteil: Prophet überspringt Zeilen mit NaN in der y-Spalte automatisch beim Fitten. Dadurch lässt sich die Ausreißerbehandlung besonders elegant gestalten: Statt verdächtige Werte zu entfernen oder zu interpolieren, setzen Sie sie auf NaN. Die Zeitstruktur bleibt erhalten, die »schmutzigen« Werte fließen nicht ins Training ein, und für die betroffenen Zeitpunkte wird trotzdem eine Prognose berechnet.

Teil C — Der 12-stufige Workflow

Der Workflow ist nicht linear, sondern iterativ aufgebaut. Zwei Rücksprungschleifen sind bewusst vorgesehen:

- Sind die Fehlermetriken in Schritt 10 unbefriedigend, kehrt man zu Schritt 5 (Saisonalitätsmodus) oder Schritt 4 (Wachstumsmodell) zurück — die Modellstruktur muss dann überdacht werden.
- Zeigen die Residuen in Schritt 11 systematische Muster, kehrt man zu Schritt 6 zurück und ergänzt zusätzliche Saisonalitäten, Feiertage oder Regressoren.

Die vier Phasen (Vorbereitung, Modell-Konfiguration, Validierung, Prognose) entsprechen den farbigen Blöcken im Workflow-Diagramm.

Phase 1 — Vorbereitung (Schritte 1 bis 3)

Schritt 1 — Daten laden

Ausgangspunkt ist eine Rohdatenquelle, typischerweise eine CSV-Datei. Ziel dieses Schritts ist es, die Daten in das von Prophet geforderte Format zu überführen: ein DataFrame mit genau den Spalten `ds` und `y`. Das bedeutet konkret:

1. Die Zeitstempel-Spalte in `ds` umbenennen und mit `pd.to_datetime()` in einen echten `datetime`-Typ konvertieren (kritisch — Prophet akzeptiert keine Strings).
2. Die Messwert-Spalte in `y` umbenennen und sicherstellen, dass sie numerisch ist.
3. Alle anderen Spalten zunächst entfernen — sie kommen, falls benötigt, erst in Schritt 6 wieder ins Spiel.
4. Einen Plausibilitätsblick werfen: `df.info()`, die ersten und letzten Zeilen, Zeitraum und Anzahl der Beobachtungen.

Schritt 2 — Sichtprüfung

Ein einfacher Linien-Plot ist erstaunlich aufschlussreich. Die menschliche Mustererkennung erkennt Strukturen, die man mit statistischen Tests mühsam nachweisen müsste. Prüfen Sie Ihre Zeitreihe systematisch auf vier Merkmale:

1. **Trend:** Gibt es eine erkennbare langfristige Richtung? Steigt oder fällt die Reihe? Folgt sie einer S-Kurve? Ist sie flach?
2. **Saisonalität:** Sind wiederkehrende Muster sichtbar? Jahreszeitlich, wöchentlich, täglich? Wie stark ist die Amplitude?
3. **Ausreißer:** Einzelne Punkte, die deutlich aus dem Muster fallen? Ursache bekannt oder zufällig?
4. **Strukturbrüche:** Abrupte Niveau- oder Dynamikwechsel — etwa infolge von Corona, Sortimentsumstellungen oder Methodenwechseln?

Die Antworten auf diese vier Fragen bestimmen wesentliche Entscheidungen in den folgenden Schritten: Logistisches Wachstum ja oder nein? Additive oder multiplikative Saisonalität? Welche Ausreißerbehandlung ist angemessen?

Faustregel zum Saisonalitätsmodus

Vergleichen Sie die Standardabweichung der ersten Hälfte der Zeitreihe mit der zweiten Hälfte. Hat sich die Schwankungsbreite deutlich vergrößert, spricht das für eine multiplikative Saisonalität — die Amplitude wächst mit dem Niveau. Bleibt sie ungefähr gleich, reicht die additive Variante.

Schritt 3 — Datenaufbereitung

Im Vergleich zu SARIMA ist hier erstaunlich wenig zu tun. Kein Differenzieren, keine Log-Transformation, keine Stationarisierung. Die einzige wirklich sinnvolle Operation ist die Ausreißerbehandlung:

1. Eine Ausreißer-Schwelle definieren — üblich ist z-Score-basiert, etwa $|z| > 3$ (also Werte weiter als 3 Standardabweichungen vom Mittelwert entfernt).
2. Diese Werte auf NaN setzen (nicht entfernen!). Die Zeitstruktur bleibt damit erhalten.
3. Die Rohdaten und die bereinigte Version im direkten Vergleich plotten, um die Auswirkung zu dokumentieren.

Prophet verarbeitet die NaNs nativ. Es wird nicht interpoliert, sondern die betroffenen Zeitpunkte werden im Training schlicht ausgelassen — das Modell lernt nur aus den sauberen Beobachtungen.

Phase 2 — Modell-Konfiguration (Schritte 4 bis 7)

Schritt 4 — Wachstumsmodell wählen

Auf Grundlage der Sichtprüfung in Schritt 2 entscheiden Sie, ob das Wachstum linear oder logistisch modelliert werden soll. Bei logistischem Wachstum müssen Sie außerdem eine Obergrenze (cap) und optional eine Untergrenze (floor) festlegen — diese werden als zusätzliche Spalten in den DataFrame geschrieben und müssen auch im Zukunfts-DataFrame gesetzt sein.

Modell	Wann geeignet?	Erfordert zusätzlich
linear	Trend steigt oder fällt ohne erkennbare Sättigungsgrenze	nichts
logistic	Wachstum flacht ab; es existiert eine natürliche Obergrenze (z. B. Marktpotenzial, Kapazität)	cap-Spalte (und optional floor)

Schritt 5 — Saisonalitätsmodus festlegen

Hier setzen Sie den Schalter »additive« oder »multiplicative«, wie in Teil A Abschnitt 2 erläutert. Die Wahl beruht auf dem Varianzvergleich aus Schritt 2. Beachten Sie: Sollten die Fehlermetriken in Schritt 10 trotz Tuning unbefriedigend bleiben, ist dies eine der ersten Stellschrauben, an denen Sie drehen sollten.

Schritt 6 — Features und Events definieren

In diesem Schritt bereichern Sie das Modell mit Domänenwissen an. Prophet bietet drei Mechanismen, die jeweils eine andere Art von Effekt adressieren:

Mechanismus	API-Aufruf	Wann einsetzen?
Zusätzliche Saisonalität	<code>add_seasonality(name, period, fourier_order)</code>	Periodische Muster, die Prophet nicht automatisch erkennt (z. B. Wochentag bei Monatsdaten)
Feiertage / Events	holidays-DataFrame	Kalendereffekte mit bekannten Daten (Weihnachten, Black Friday, Schulferien)
Externe Regressoren	<code>add_regressor(name)</code>	Erklärende Variablen ohne feste Periode (Temperatur, Werbebudget, Konjunkturindex)

Nicht jeder Datensatz braucht alle drei Mechanismen — viele Reihen kommen mit den Default-Saisonalitäten (Jahres- und Wochensaison) aus. Ergänzen Sie hier gezielt, wenn die Residuenanalyse in Schritt 11 entsprechende Hinweise liefert.

Schritt 7 — Modell fitten

Nun wird das Prophet-Objekt mit allen getroffenen Entscheidungen konfiguriert und auf die Trainingsdaten angepasst. Der Methodenaufwurf hat typischerweise die Form:

```
modell = Prophet(growth=..., seasonality_mode=..., ...)
modell.add_seasonality(...)      # optional
modell.add_regressor(...)       # optional
modell.fit(df_train)
```

Unter der Haube führt Prophet eine bayesianische MAP-Schätzung (Maximum a Posteriori) über das Stan-Framework durch. Für Studierende reicht es hier zu wissen: Das Modell lernt alle Koeffizienten gemeinsam, und bei typischen Datensätzen ist das Training in wenigen Sekunden abgeschlossen.

Phase 3 — Validierung und Diagnose (Schritte 8 bis 11)

Schritt 8 — Komponentenanalyse

Der Aufruf `modell.plot_components()` zeichnet jede geschätzte Komponente einzeln: Trend, Jahressaisonalität, Wochensaisonalität, Feiertage, jeder Regressor. Prüfen Sie:

- Sieht der Trend-Verlauf plausibel aus? Entspricht er der visuellen Einschätzung aus Schritt 2?
- Stimmt die Wellenform der Saisonalitäten mit dem Domänenwissen überein? (Etwa: Passagierzahlen sollten im Sommer höher sein als im Winter.)
- Gibt es sprunghafte Trendwechsel? Dann eventuell `n_changepoints` reduzieren oder `changepoint_prior_scale` verringern.

Diese visuelle Diagnose ist in SARIMA so nicht möglich — sie ist einer der großen pädagogischen Vorzüge von Prophet. Die Studierenden sehen unmittelbar, welche Bestandteile das Modell aus den Daten extrahiert hat.

Schritt 9 — Cross-Validation und Hyperparameter-Tuning

Hier kommt das methodische Schwergewicht des Workflows. Statt die Hyperparameter — insbesondere `changepoint_prior_scale` und `seasonality_prior_scale` — zu raten, durchsuchen wir systematisch ein Parametergitter per Kreuzvalidierung. Für jede Parameterkombination wird eine Rolling-Origin-CV durchgeführt:

1. Man beginnt mit einem Anfangsteil der Daten, dessen Länge mit `initial` vorgegeben wird.
2. Auf diesem Teil wird das Modell trainiert und um `horizon` Zeitschritte in die Zukunft prognostiziert.
3. Die Prognose wird mit den tatsächlich beobachteten Werten im Prognosehorizont verglichen.
4. Das Zeitfenster wird um `period` vorgeschoben, das Modell neu trainiert — und so weiter, bis das Ende der Zeitreihe erreicht ist.
5. Die mittleren Fehler über alle CV-Durchläufe ergeben die Bewertung dieser Parameterkombination.

Am Ende gewinnt die Kombination mit dem niedrigsten Fehler (typischerweise MAPE oder RMSE); Prophet wird dann mit diesen optimalen Parametern auf dem gesamten Datensatz neu angepasst.

Faustregeln für die CV-Fenster

initial sollte mindestens doppelt so groß sein wie die längste saisonale Periode — damit in der ersten Trainingsmenge bereits mehrere vollständige Saisonzyklen enthalten sind.

horizon wird typischerweise auf die Länge gesetzt, die Sie später tatsächlich prognostizieren wollen — oft ein saisonaler Zyklus.

period ist der Abstand zwischen zwei CV-Durchläufen; übliche Wahl ist etwa die Hälfte von horizon.

Schritt 10 — Metriken auswerten

Aus der Cross-Validation ergeben sich die Standard-Fehlermetriken. Sie sollten alle drei zusammen betrachten, da sie unterschiedliche Eigenschaften beleuchten:

Metrik	Berechnung (verbal)	Was sie aussagt
MAE	Mittelwert der absoluten Differenzen $ y - \hat{y} $	Durchschnittliche Abweichung in den Einheiten der Zielgröße — leicht interpretierbar
RMSE	Wurzel aus dem Mittel der quadrierten Differenzen	Bestraft große Abweichungen stärker; empfindlich gegenüber Ausreißern
MAPE	Mittlerer absoluter prozentualer Fehler	Skalenunabhängig in Prozent; problematisch bei Werten nahe Null

Die Bewertung »akzeptabel« hängt immer vom Anwendungsfall ab. Ein MAPE von 5 % ist für tägliche Aktienkurse unrealistisch gut, für monatliche Stromverbräuche solide. Wichtig ist der Vergleich mit einem einfachen Referenzmodell (etwa: »die nächste Monatszahl entspricht dem gleichen Monat des Vorjahres«). Erst wenn Ihr Prophet-Modell diesen Naiv-Benchmark schlägt, bringt es tatsächlich Mehrwert.

Sind die Metriken trotz Tuning unbefriedigend, verlässt der Workflow den Validierungspfad und springt zurück zu Schritt 5 (oder sogar Schritt 4): Die strukturelle Modellentscheidung war vermutlich falsch.

Schritt 11 — Residuenanalyse

Die Residuen $r(t) = y(t) - \hat{y}(t)$ sind die Fehler des Modells an jedem Trainingspunkt. Idealerweise wirken sie wie zufälliges Rauschen ohne sichtbares Muster. Finden sich hingegen systematische Strukturen in den Residuen — etwa ein mehrjähriger Schwingungsrhythmus, eine wochentagsabhängige Abweichung oder eine Korrelation mit einer externen Größe — dann hat das Modell eine Komponente übersehen.

In diesem Fall ist die Konsequenz klar: zurück zu Schritt 6 und die fehlende Komponente ergänzen — etwa eine zusätzliche Saisonalität, einen Feiertags-Kalender oder einen externen Regressor.

Phase 4 — Prognose (Schritt 12)

Schritt 12 — Prognose mit Konfidenzintervall

Der letzte Schritt ist technisch schlicht. Mit `modell.make_future_dataframe(periods=..., freq=...)` erzeugen Sie einen DataFrame, der die bekannten Zeitpunkte plus die gewünschten zukünftigen Zeitpunkte enthält. Bei logistischem Wachstum müssen `cap` und `floor` auch für diese Zukunftszeilen gesetzt werden. Externe Regressoren müssen mit zukünftigen Werten befüllt sein.

Der Aufruf `modell.predict(future)` liefert pro Zeitpunkt drei relevante Werte zurück:

- **yhat:** Punktprognose (Erwartungswert des Modells)
- **yhat_lower:** untere Grenze des 80 %-Konfidenzintervalls
- **yhat_upper:** obere Grenze des 80 %-Konfidenzintervalls

Das Konfidenzintervall ist das wichtigste Ergebnis neben der Punktprognose. Es drückt aus, wie unsicher das Modell bezüglich der Vorhersage ist. Typischerweise weitet sich das Intervall mit zunehmendem Prognosehorizont — die Unsicherheit wächst, je weiter man in die Zukunft schaut.

Ein methodischer Hinweis, den Sie den Studierenden mitgeben sollten: Eine Prognose ist kein Orakel, sondern eine probabilistische Aussage. Die Kommunikation der Unsicherheit ist ein integraler Bestandteil der Prognose — ein Punktwert ohne Intervall ist eine halbe Antwort.

Teil D — Zusammenfassung

Entscheidungen im Überblick

Die folgende Tabelle listet die wesentlichen Stellschrauben des Workflows auf — welche Entscheidung in welchem Schritt zu treffen ist, und welche Variable im Notebook dafür gesetzt wird.

Entscheidung	Notebook-Variable	Schritt	Grundlage der Entscheidung
Datensatz	DATENSATZ	1	Gegebene Aufgabenstellung
Ausreißer-Schwelle	SCHWELLE	3	Verteilungsanalyse in Schritt 2
Wachstumsmodell	GROWTH	4	Sättigungsgrenze bekannt?
Saisonalitätsmodus	SEASONALITY_MODE	5	Varianzvergleich erste vs. zweite Hälfte
Zusätzliche Saisonalitäten	EXTRA_SEASONALITIES	6	Residuenmuster, Domänenwissen
Feiertage	feiertage	6	Kalender des Anwendungsbereichs
Externe Regressoren	EXTRA_REGRESSORS	6	Kausale Hypothesen
Changepoint-Prior	(automatisch via Grid Search)	9	Cross-Validation
Saisonalitäts-Prior	(automatisch via Grid Search)	9	Cross-Validation
Prognosehorizont	HORIZONT	12	Fachliche Anforderung

Iterationslogik

Der Workflow kennt zwei Rücksprung-Muster, die beide eine klare didaktische Botschaft tragen:

- **Metriken unzureichend in Schritt 10** → **zurück zu Schritt 5 (oder 4)**. Wenn auch systematisches Tuning nicht hilft, liegt der Fehler nicht im Detail, sondern in der Modellstruktur selbst.
- **Muster in den Residuen in Schritt 11** → **zurück zu Schritt 6**. Eine erklärbare Systematik in den Residuen ist ein Signal, dass dem Modell eine Einflussgröße fehlt — kein Parameterproblem, sondern ein Feature-Problem.

Abschluss

Prophet nimmt der Anwenderin nicht die inhaltliche Arbeit ab. Die Entscheidungen zu Wachstum, Saisonalitätsmodus und Features sind inhaltliche — sie beruhen auf dem Blick in die Daten, auf Domänenwissen und auf der Bereitschaft, Modellentwürfe kritisch zu hinterfragen. Was Prophet tut, ist die technische Umsetzung dieser Entscheidungen zu vereinfachen und den Anwender bei der Diagnose zu unterstützen, indem das Modell seine einzelnen Komponenten sichtbar macht.

Dieser strukturierte 12-Schritte-Workflow ist kein Rezept, das mechanisch abgearbeitet werden müsste, sondern ein Gerüst, das sicherstellt, dass die wesentlichen Fragen an geeigneter Stelle gestellt und die Ergebnisse systematisch überprüft werden. Damit ist er ein didaktisches Werkzeug ebenso wie ein praktisches — und genau in dieser Doppelrolle sollten Sie ihn Ihren Studierenden vermitteln.